ГУАП

КАФЕДРА № [ВПИСАТЬ КАФЕДРУ]

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

_____	_____	_____
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ № 1

Инструментальный контур и воспроизводимое исполнение минимального программного компонента

по МДК.01.04 «Системное программирование»

РАБОТУ ВЫПОЛНИЛ
Студент гр. № [ВПИСАТЬ ГРУППУ] _____ _____ _____
подпись, дата Д. Карачебан

Санкт-Петербург

2026

ВВЕДЕНИЕ

Цель работы — подготовить исполняемый компонент проекта HitPipe и проследить весь путь от исходного кода до проверенного запуска. На индивидуальном участке реализована проверка пригодности существующей прикладной сессии. Одна и та же логика используется прикладным потребителем и диагностической командой.

Задачи работы: выбрать и описать инструменты, собрать компонент в двух режимах, исследовать исполняемый файл, проверить конфигурацию и диагностику, воспроизвести запуск другого компонента и показать, что автоматическая проверка действительно обнаруживает нарушенное правило.

1 СВЕДЕНИЯ О РАБОТЕ И ПРОЕКТЕ

Поле	**Значение**
Автор	Дмитрий Карачебан
Проект	HitPipe — сопровождение бизнес-операций между цифровыми сервисами
Индивидуальный участок	IAM: проверка прикладной сессии; каталог services/iam
Роль в команде	Руководитель проекта; аутентификация, авторизация и профиль
Рецензент	Тимофей Патрушев
Период работы	18–19 октября 2026 года
Репозиторий	hitpipe/hitpipe
Интегрированная версия	main, commit 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00; PR №12–19 объединены 19.10.2026

2 ЦЕЛЬ, ПРОЕКТ И СОБСТВЕННАЯ ЧАСТЬ

HitPipe сопровождает бизнес-операцию от входного события до установленного результата либо понятного ожидания или исключения. Для работы оператора требуется действующая прикладная сессия. IAM определяет, известна ли сессия, не отозвана ли она и не истекли ли её сроки. При этом действующая сессия сама по себе не разрешает бизнес-действие: проверка роли и текущих полномочий выполняется отдельно.

Индивидуальный результат находится в services/iam. Это самостоятельный Go-модуль с чистой предметной логикой, прикладным сценарием, подменяемыми зависимостями и диагностическим исполняемым файлом. Команда работает только с синтетическими входными данными и не обращается к БД, платёжным сервисам или языковой модели.

2.1 Семь позиций инструментального контура

Позиция	**Средство, назначение и применение**
Сборка и преобразование	Go 1.27.2: проверка типов, компиляция и линковка. Python-команда workspace.py вызывает сборку конкретного сервиса; результат — ELF-файл linux/amd64. Сборка dev и release выполнена.
Управление зависимостями	Go modules; версия языка закреплена в services/iam/go.mod. Участок использует стандартную библиотеку, прямых сторонних Go-модулей нет. Состав зависимостей проверен командой go list.
Автоматические тесты	Пакет testing, табличные и граничные тесты, fuzz-проверки, отчёт покрытия и отдельный запуск мутаций. Проверяются значения результатов, ошибки и отсутствие утечек.

Позиция	Средство, назначение и применение
Статический анализ	go vet и проверка направлений импортов в workspace.py. Domain не импортирует транспорт, хранилище или адаптеры. Проверки выполнены локально и в CI.
Динамический анализ	Детектор гонок Go, включённый через PR12_RACE=1, и запуск настоящего бинарника вне каталога исходников. Проверенные сценарии завершились успешно.
Профилирование	Предусмотрены benchmark, pprof и trace после появления представительной нагрузки реального источника сессий. На данном этапе профиль CPU/памяти не снимался; выводов о производительности нет.
Непрерывная интеграция	GitHub Actions, Ubuntu 24.04: получение исходников, Go 1.27.2, проверка источников, vet, тесты и race, dev/release, покрытие, мутации и сохранение артефактов. Workflow — .github/workflows/pr12-dk.yml.

Среда выполнения проверок: Linux amd64, Go 1.27.2, Python 3.14.4, Git 2.53.0 и GNU readelf 2.46. Отсутствие go.sum объясняется отсутствием сторонних модулей, а не незакреплёнными зависимостями. Средства подготовки отчётов не входят в зависимости исполняемого компонента.

Общее решение LAB-STACK-001 определяет технологический контур проекта. Для данного участка Go выбран за компилируемую модель исполнения, статическую типизацию, стандартные средства тестирования и возможность получить самостоятельный исполняемый файл. Отдельная интерпретируемая учебная программа означала бы дублирование предметного правила; вместо неё создан модуль, пригодный для дальнейшего развития IAM.

3 ХОД РАБОТЫ

3.1 Подготовка собственного каркаса

Работа разделена на связанные изменения: HIT-19 — предметная проверка и диагностика, HIT-20 — прикладной потребитель и адаптеры, HIT-21 — расширенная система проверок. Каждому изменению соответствует отдельный PR №14, №15 и №16. Общий каркас проекта использован как основание; исходники других сервисов в рамках этих задач не переписывались.

В опубликованном техническом ревью Тимофея Патрушева код PR №12 –18 получил положительную оценку в пределах рассмотренного участка. Два замечания к отчётному пакету — каталог запуска Go-команд и перенос строк таблиц — исправлены. Изменения объединены в основную ветку; обязательные проверки IAM, Intake, Processing и Delivery завершились успешно.

На рисунке 1 приведены сведения об изменении HIT-19 и его интеграции. История задач, исходных веток и проверок сохранена в GitHub.

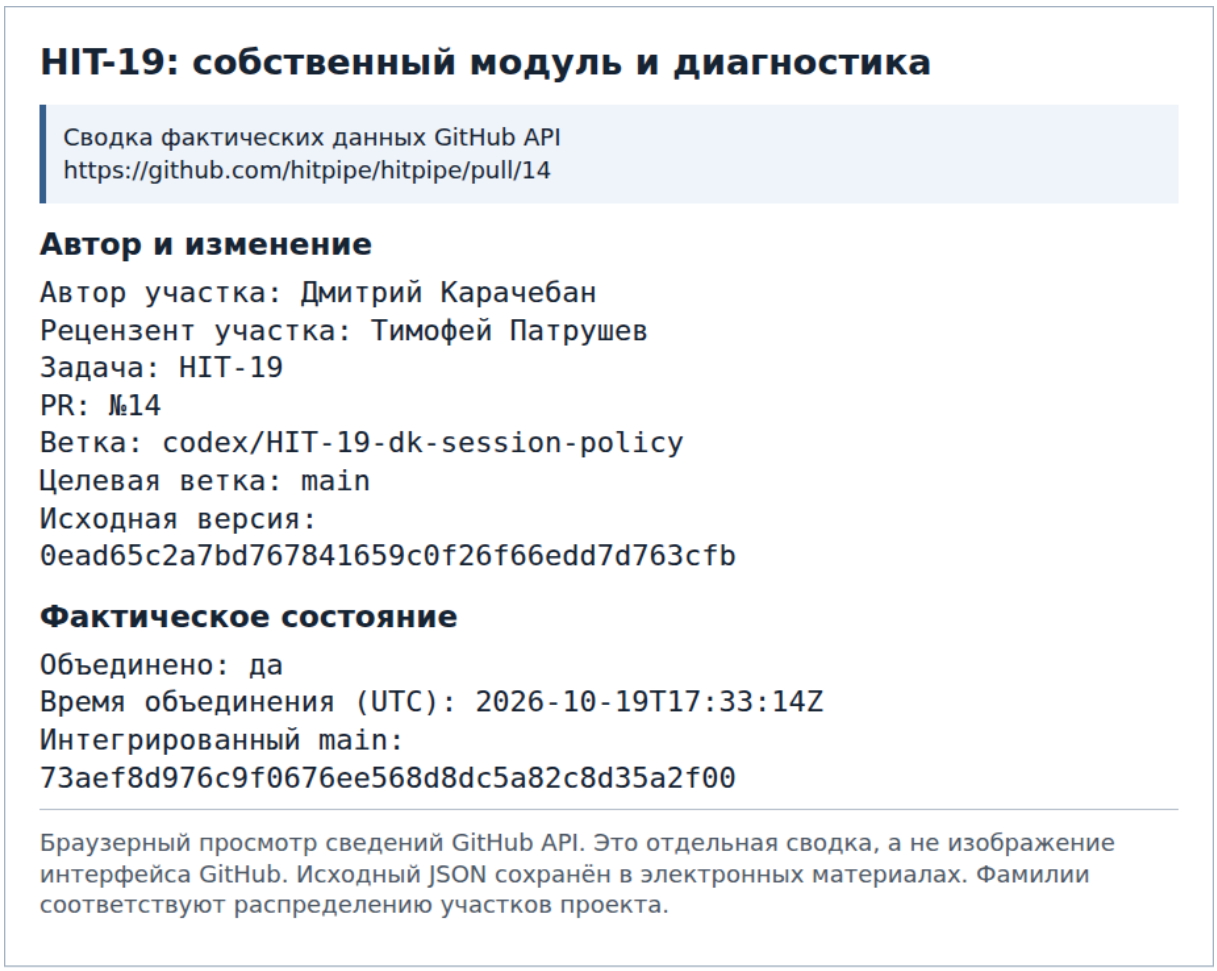


Рисунок 1 - Собственная часть исполняемого каркаса

3.2 Путь исполнения и состав артефакта

Путь исполнения имеет следующий вид: исходники Go и go.mod → проверка типов → компиляция пакетов → линковка со средой выполнения Go → ELF-файл → запуск процессом ОС → main и сборка зависимостей → чтение входных данных → Lookup → Clock → Evaluate → безопасный результат и код завершения.

Python, Git и инструменты сборки нужны для подготовки и проверки, но не для исполнения готового бинарника. JSON-файл передаётся отдельно. Проверки file, go version -m и readelf показали статический ELF64 x86-64, наличие .debug_info и отсутствие динамической секции у исследованных сборок. Значение CGO_ENABLED=1 в метаданных само по себе не означает динамическую линковку именно этого файла.

3.3 Сравнение dev и release

Обе сборки изготовлены из одной версии исходников 2cd7c032e8ede98c75181599b440e97657bff197 при чистом рабочем дереве. Для сравнения использован один и тот же valid.json. Исходники IAM и его тестов в этой версии и интегрированном main совпадают.

Свойство	**Основные параметры**
Режим dev	-gcflags=all=-N -l; метаданные линковщика
Режим release	-trimpath; метаданные линковщика

Свойство	**Наблюдённый размер**
Режим dev	4 622 983 байта
Режим release	4 409 836 байт

Свойство	**Формат**
Режим dev	Статический ELF64 x86-64
Режим release	Статический ELF64 x86-64

Свойство	**Отладочная информация**
Режим dev	.debug_info присутствует; файл не stripped
Режим release	.debug_info присутствует; файл не stripped

Свойство	**Метаданные --version**
Режим dev	Тот же commit, dirty=false, buildMode=dev
Режим release	Тот же commit, dirty=false, buildMode=release

Свойство	**Штатный сценарий**
Режим dev	session_valid, код 0
Режим release	session_valid, код 0

Свойство	**Запуск вне исходников**
Режим dev	Копия бинарника и входных файлов: успешно
Режим release	Копия бинарника и входных файлов: успешно

Разница размеров является наблюдением для конкретных сборок. Она не доказывает ускорения release. В выбранном release-профиле отладочные символы не удаляются. Повторный запуск выполнялся из временного каталога отдельной копией файла, а не через go run.

Подготовка IAM: dev и release

> Версия исходников: 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00
> Время UTC: 2026-10-19T17:39:19.724844+00:00
> $ python3 tools/pr12/workspace.py build iam --mode dev --require
> Фактический exit: 0
> $ python3 tools/pr12/workspace.py build iam --mode release --require
> Фактический exit: 0

Версии реальных бинарников

```
version=pr12 commit=73aef8d976c9f0676ee568d8dc5a82c8d35a2f00
dirty=false buildMode=dev
iam-contractcheck event=start
iam-contractcheck event=end status=version

version=pr12 commit=73aef8d976c9f0676ee568d8dc5a82c8d35a2f00
dirty=false buildMode=release
iam-contractcheck event=start
iam-contractcheck event=end status=version
```

Тип и состав

```
.pr12-build/iam/dev/iam-contractcheck:     ELF 64-bit LSB
executable, x86-64, version 1 (SYSV), statically linked, Go
BuildID=HTmAD8mIlosghQZpDOtT/hu73-X5fJNYuzf4SjO-
a/4bdvkFdAKxAITC7mMZba/xz8ntYIzcaMWT0lldKYU,
BuildID[sha1]=8b84616c84d3295066b1e439f68fd3ff2e5a10b0, with
debug_info, not stripped
.pr12-build/iam/release/iam-contractcheck: ELF 64-bit LSB
executable, x86-64, version 1 (SYSV), statically linked, Go
BuildID=kbI87MKa6pqD4pLMytEj/ptVC38XCzij7393YpChh/kIuwH3GnbebC5vO
CUZ4M/k3RsO3lS_p4A3JuoY3rZ,
BuildID[sha1]=58f1d6cdd4ac2f16dd20bee7d7368834c5be156a, with
debug_info, not stripped
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете.

Рисунок 2 - Подготовка одного компонента dev/release

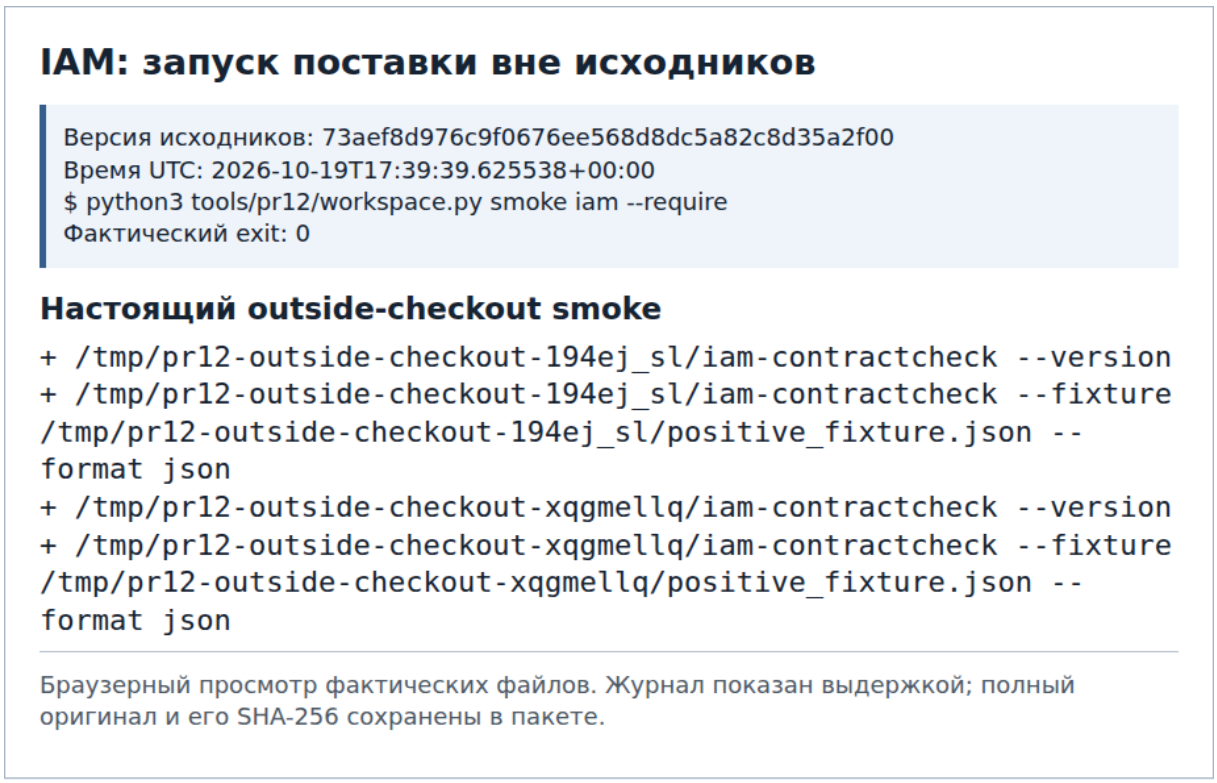


Рисунок 3 - Поставка запущена вне исходников

3.4 Конфигурация, остановка и повторный запуск

Конфигурация задаёт путь к синтетическому входу, формат ответа и выбор адаптеров. Корректный запрещающий результат — например, истёкший срок — является обработанным бизнес-исходом и имеет код завершения 0. Неверный JSON или неизвестный формат дают код 2. Недоступная зависимость даёт код 3.

Команда выполняет один сценарий и завершается. Она не хранит состояние между запусками и не меняет исходный JSON. Повторный запуск с теми же значениями и фиксированным временем даёт тот же результат. После ошибочного файла восстановление состоит в выборе корректного входа; очистка внешних ресурсов не требуется.

4 РЕЗУЛЬТАТЫ

4.1 Проверки компонента

Сценарий	**Ожидание и фактический результат**
--version	Выведены версия исходников, признак dirty и режим сборки; проверка успешна в dev и release.
valid.json, text/json	session_valid, код 0; присутствуют начальная и конечная диагностические записи.
idle-equal.json	denied/idle_expired, код 0: равенство срока означает истечение.
malformed.json и неверный формат	invalid_config, код 2; содержимое входа не раскрывается.
--lookup file/fixed	Тот же потребитель получает одинаковый результат при одинаковых данных.
--clock system без подтверждения исправности часов	unavailable/clock_unavailable, код 3. Неподтверждённое системное время не считается доверенным.
Безопасность вывода	Тестовые секретные строки, идентификатор сессии, профиль и тело входного файла отсутствуют в stdout/stderr.



Рисунок 4 - Штатный диагностический запуск

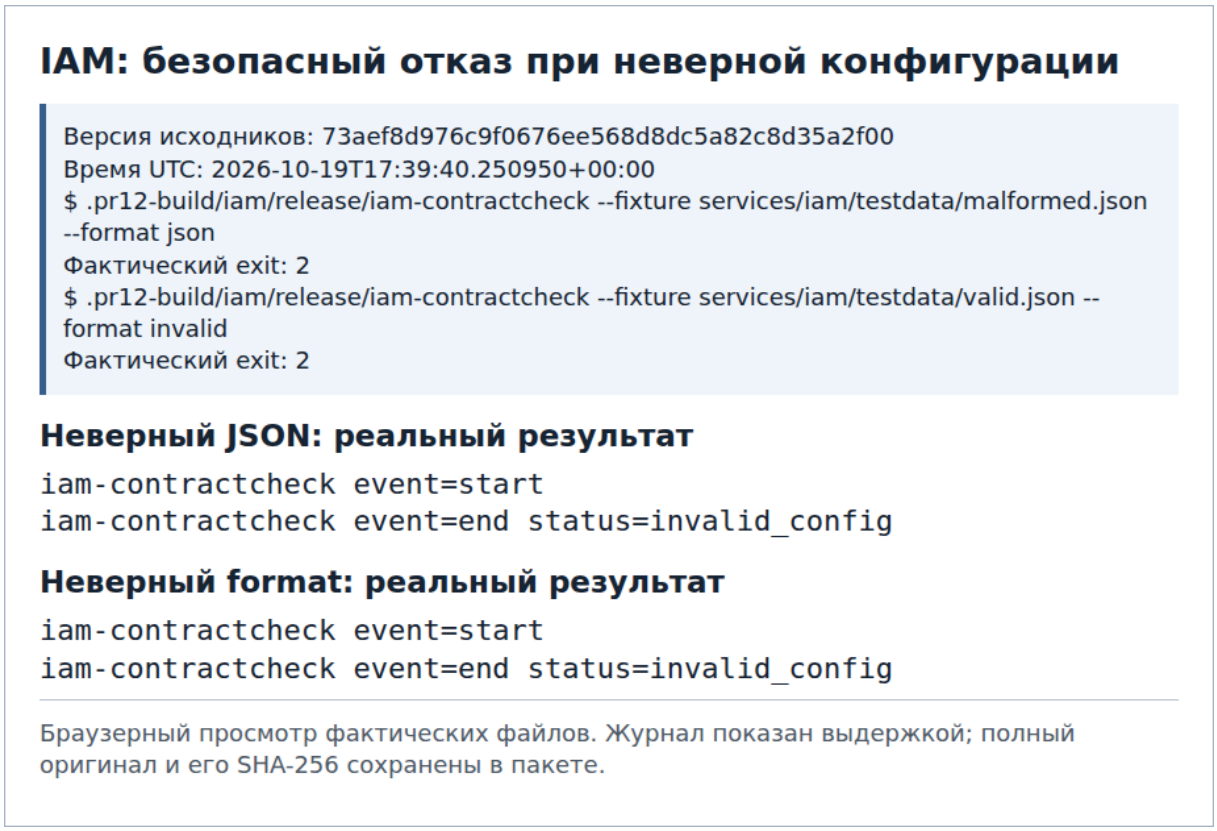


Рисунок 5 - Неверная конфигурация и безопасная диагностика

4.2 Техническое воспроизведение компонента другого участника

19 октября 2026 года на интегрированной версии 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00 выполнено техническое

воспроизведение компонента Processing, за который отвечает Тимофей Патрушев. Использовано отдельное чистое рабочее дерево репозитория; проверка не требует ключей внешних сервисов или персональных данных.

Из корня репозитория выполнены проверка Processing с детектором гонок, диагностический запуск в режимах dev/release, тест TestReaderSubstitution и получение списка импортов. Все четыре команды завершились с кодом 0. Для File-реализации и тестовой подмены SourceFactsReader проверены штатный, ошибочный и граничные случаи. Карта зависимостей сопоставлена с фактическими импортами; циклов не обнаружено. Полные команды, параметры среды, время и результаты сохранены в evidence/peer-technical.

На рисунке 6 показаны результаты проверки и запуска Processing. Иллюстрация представляет браузерный просмотр фактических журналов; полные файлы сохранены отдельно.

Чужой компонент Processing: техническое воспроизведение

> Версия исходников: 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00

Проверки и race

```
ok
github.com/hitpipe/hitpipe/services/processing/cmd/processing-
contractcheck   0.006s  coverage: 90.6% of statements
ok
github.com/hitpipe/hitpipe/services/processing/internal/adapters/
fixtures/input  0.016s  coverage: 85.7% of statements
ok
github.com/hitpipe/hitpipe/services/processing/internal/adapters/
fixtures/sourcefacts    0.003s  coverage: 87.5% of statements
ok
github.com/hitpipe/hitpipe/services/processing/internal/applicati
on/evaluatepaidorder    0.005s  coverage: 100.0% of statements
ok
github.com/hitpipe/hitpipe/services/processing/internal/domain/el
igibility       0.005s  coverage: 100.0% of statements
+ go test -race -count=1 ./...
ok
github.com/hitpipe/hitpipe/services/processing/cmd/processing-
contractcheck   1.016s
ok
```

Рисунок 6 - Техническое воспроизведение компонента Processing (часть 1 из 2)

```
ok
github.com/hitpipe/hitpipe/services/processing/internal/adapters/
fixtures/input  1.050s
ok
github.com/hitpipe/hitpipe/services/processing/internal/adapters/
fixtures/sourcefacts    1.015s
ok
github.com/hitpipe/hitpipe/services/processing/internal/applicati
on/evaluatepaidorder    1.025s
ok
github.com/hitpipe/hitpipe/services/processing/internal/domain/el
igibility       1.016s
```

Поставка вне исходников

```
+ /tmp/pr12-outside-checkout-gto0ksjl/processing-contractcheck --
version
+ /tmp/pr12-outside-checkout-gto0ksjl/processing-contractcheck --
fixture /tmp/pr12-outside-checkout-gto0ksjl/positive_fixture.json
--format json
+ /tmp/pr12-outside-checkout-l0rsi05d/processing-contractcheck --
version
+ /tmp/pr12-outside-checkout-l0rsi05d/processing-contractcheck --
fixture /tmp/pr12-outside-checkout-l0rsi05d/positive_fixture.json
--format json
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете.

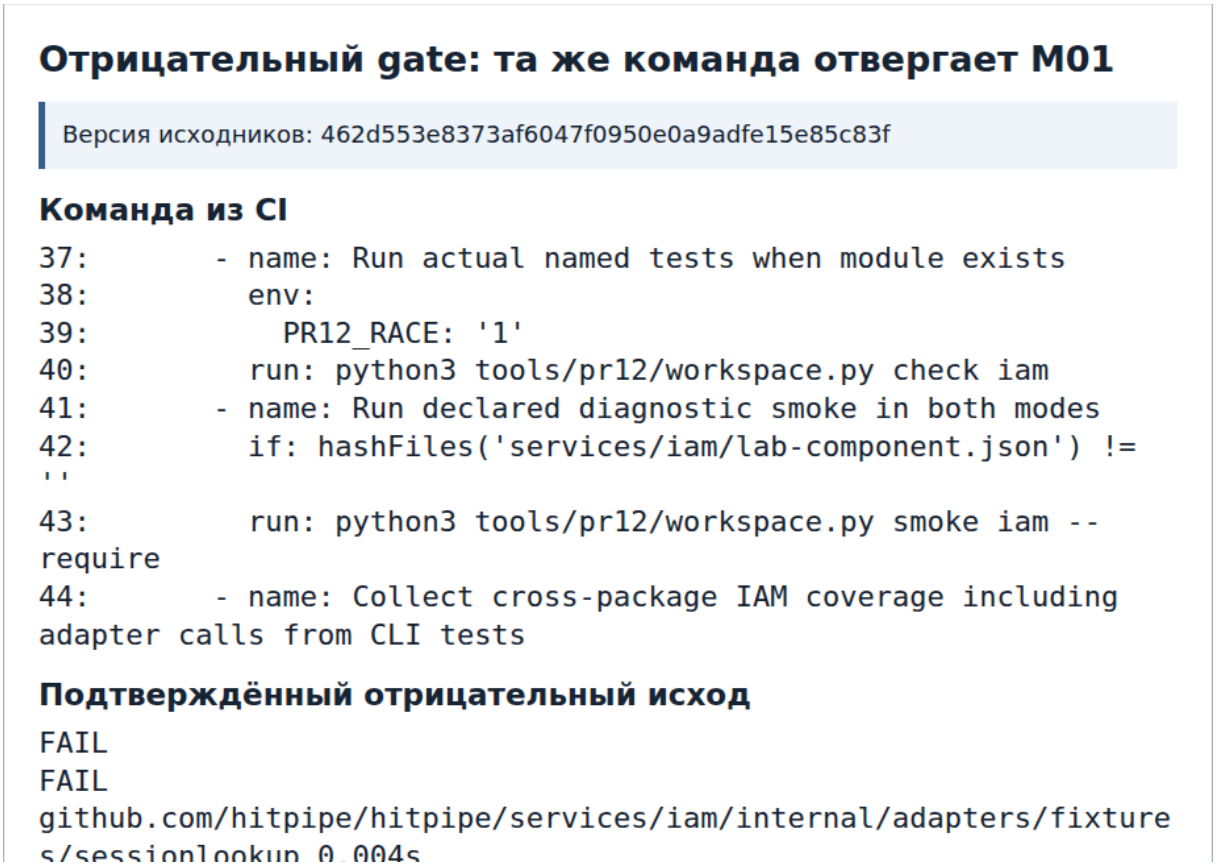
Рисунок 6 - Техническое воспроизведение компонента Processing (часть 2 из 2)

4.3 Автоматический контроль с отрицательной демонстрацией

Workflow pr12-iam проверяет не только компиляцию, но и выполнение именованных тестов, отсутствие запрещённых зависимостей, запуск бинарника и поведение при нарушении правил. Успешный запуск проверенной версии — run 39177171258.

Для отрицательной демонстрации в отдельном временном рабочем дереве изменено сравнение срока бездействия: вместо now>=deadline использовано now>deadline. Мутант M01 компилируется с кодом 0, но та же команда проверки завершается с кодом 1: тест равенства срока обнаруживает ошибку. После восстановления исходного файла проверка вновь завершается

с кодом 0. Таким образом, успешная компиляция не маскирует нарушение предметного правила.



Отрицательный gate: та же команда отвергает M01

> Версия исходников: 462d553e8373af6047f0950e0a9adfe15e85c83f

Команда из CI

```
37:       - name: Run actual named tests when module exists
38:         env:
39:           PR12_RACE: '1'
40:         run: python3 tools/pr12/workspace.py check iam
41:       - name: Run declared diagnostic smoke in both modes
42:         if: hashFiles('services/iam/lab-component.json') !=
''
43:         run: python3 tools/pr12/workspace.py smoke iam --
require
44:       - name: Collect cross-package IAM coverage including
adapter calls from CLI tests
```

Подтверждённый отрицательный исход

```
FAIL
FAIL
github.com/hitpipe/hitpipe/services/iam/internal/adapters/fixture
s/sessionlookup 0.004s
```

Рисунок 7 - Та же команда CI отвергает нарушенное условие (часть 1 из 2)

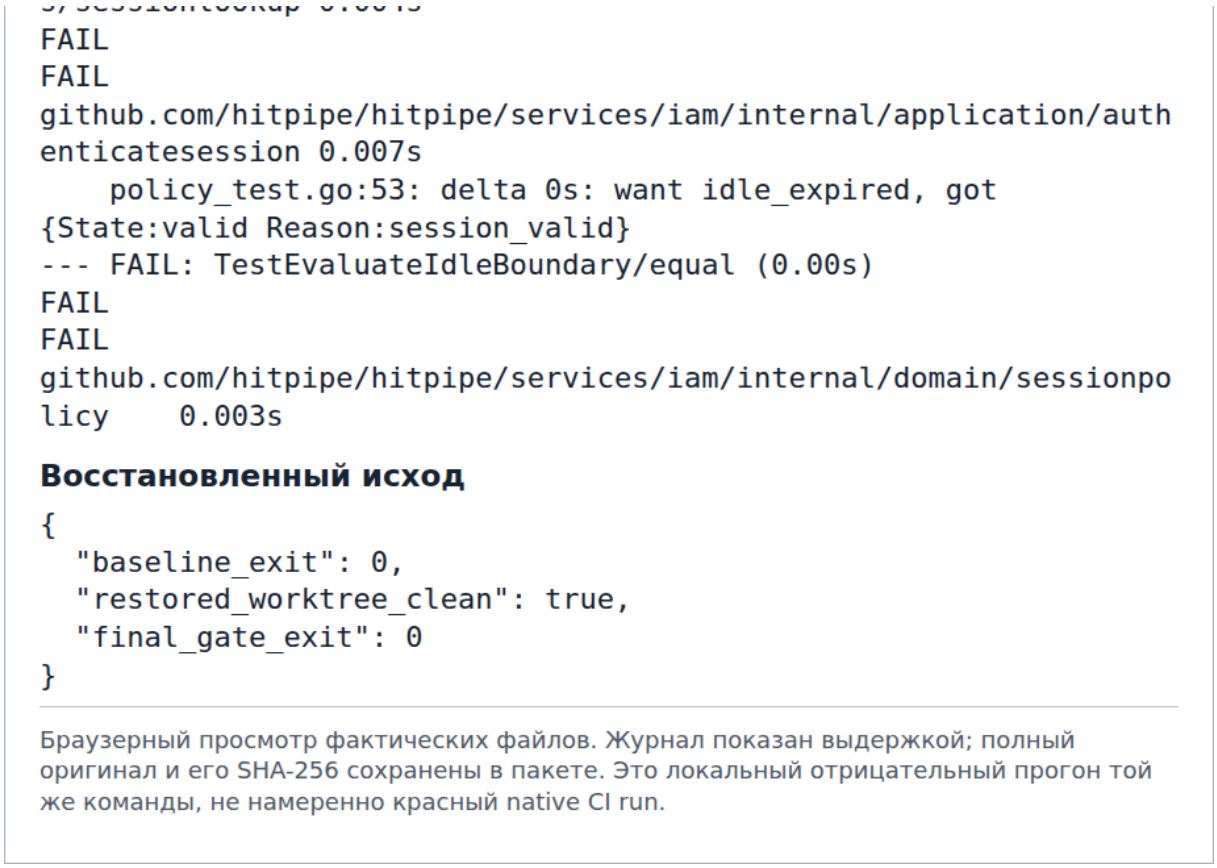


Рисунок 7 - Та же команда CI отвергает нарушенное условие (часть 2 из 2)

5 ЗАКЛЮЧЕНИЕ И ИНДИВИДУАЛЬНЫЙ ВКЛАД

Индивидуальный участок Дмитрия Карачебана включает исполняемый модуль IAM, диагностическую команду, метаданные версии, безопасную обработку конфигурации и автоматическую проверку. Описаны все семь позиций инструментального контура; для профилирования определено место в следующем этапе работы.

Практически исследованы две сборки, выполнен запуск вне каталога исходников, проверены штатные и ошибочные исходы. Отрицательный опыт подтвердил чувствительность автоматического контроля. Результат пригоден для дальнейшего подключения настоящего источника сессий без переноса предметного правила в транспортный код.

СПИСОК ИСПОЛЬЗОВАННЫХ МАТЕРИАЛОВ

1. Методические указания к практическим работам №1 «Подготовка инструментального контура программного проекта» и №2 «Модульная архитектура» по МДК01.04. Актуальные редакции, предоставленные преподавателем.

2. Методические указания к практическим работам №1 «Требования и регламент» и №2 «Проектирование и реализация системы проверок программного продукта» по МДК03.02. Актуальные редакции, предоставленные преподавателем.

3. Проект HitPipe: описание MVP, регламент изменений, решение о технологическом стеке LAB-STACK-001; архитектурные материалы HP-BP-001, разделы T/002, T/003, T/013, и HP-BP-007, раздел T/005. Копии исходных документов и контрольные суммы — docs/labs/pr12/upstream и source-files.json.

4. Исходный код и документация IAM: module-contract.md, policy.go, service.go, адаптеры, диагностическая команда и тесты.

5. Материалы индивидуального участка Дмитрия Карачебана: требования, карта модулей, DK-ADR-001, проектирование тестов, результаты и реестр рисков.

6. Изменение требований, PR №13, реализация модуля, PR №14, итоги технического ревью, PR №19.

7. Успешный запуск CI для проверенной версии IAM. Журналы повторных запусков на интегрированной версии и исходные изображения включены в электронные материалы работы.